

VITyarthi – Build Your Own Project

COMPUTER VISION PROJECT REPORT

Intelligent Panorama Builder

A classical computer vision system for combining overlapping images into a panoramic view.

Submitted by: Arpit Gupta

Reg. no. : 24BAI10289

Course: Computer Vision

Course code : CSE3010

Academic Project

1. Introduction

A panorama is created by combining several photographs that cover different parts of the same scene. Doing this manually is possible, but it becomes difficult when the images have different positions, slight rotations, or changes in perspective. The purpose of this project was to build a small computer vision application that performs this process automatically.

The Intelligent Panorama Builder takes multiple images as input, finds visual features shared between the images, estimates how the images relate to each other, warps them into a common coordinate system, and blends the overlapping areas. It also checks whether the images are related enough to produce a reasonable result. If the images have very little usable overlap, the program reports an error instead of forcing an unreliable panorama.

2. Problem Statement

Given a set of overlapping photographs, the system should determine their relative alignment and combine them into one larger view. The main practical problem is that input images are not always supplied in the correct order and some images may not contain enough common visual information. The project therefore uses feature matching, RANSAC-based homography estimation, validation, automatic ordering, warping and blending.

3. Objectives

- Accept two or more images through a command-line interface or GUI.
- Detect useful visual features using SIFT or ORB.
- Match features between related images and estimate a homography using RANSAC.
- Automatically determine a usable image order when possible.
- Reject clearly unrelated image pairs rather than producing meaningless output.
- Warp and blend accepted images into a single panorama.
- Keep memory usage under control for larger inputs.
- Provide useful errors and diagnostics when stitching cannot be completed.

4. Functional Requirements

Image Input Module: Accept multiple image files directly or through a directory/GUI selection.

Feature Detection and Matching Module: Extract SIFT/ORB features and find corresponding points between images.

Homography and Validation Module: Estimate the geometric transformation using RANSAC and check the quality of the result.

Automatic Ordering Module: Use pairwise matching information to determine a practical sequence of images.

Warping and Canvas Module: Project accepted images into a common panorama coordinate system while checking canvas size.

Blending and Output Module: Combine overlapping regions and save the final panorama.

5. Non-Functional Requirements

- Usability: commands and GUI selection should be simple enough for a user to run without changing source code.
- Reliability: invalid files, weak matches and unsafe transformations should be handled with clear errors.
- Performance: feature matching and image processing should avoid unnecessary repeated work where possible.
- Resource efficiency: very large projected canvases should be detected before an excessive memory allocation is attempted.
- Maintainability: the project is divided into separate modules for matching, homography, ordering, stitching and tests.
- Error handling: the application should fail safely when the images do not contain enough reliable overlap.

6. System Architecture

The system follows a modular pipeline. Input images first go through feature detection and matching. The resulting correspondences are used for RANSAC homography estimation. Valid pairwise transformations are then used by the ordering and stitching stages. Finally, the images are warped into a shared canvas, blended and cropped before being written to disk.

Input Images → Feature Detection → Feature Matching → RANSAC Homography → Geometry Validation → Image Ordering → Homography Chaining → Warping → Blending/Cropping → Panorama Output

7. Workflow

1. Select two or more input photographs.
2. Load and preprocess the images.
3. Detect local features and descriptors.
4. Match descriptors between image pairs.
5. Estimate the homography with RANSAC.
6. Check inlier ratio, reprojection error and spatial distribution of inliers.
7. Build an image order from validated pairwise relationships when automatic ordering is enabled.
8. Transform the images into a common coordinate frame.
9. Allocate a safe panorama canvas.
10. Blend overlapping regions and crop unnecessary borders.
11. Save the panorama or show an informative error if the data is not suitable.

8. Core Computer Vision Methods

8.1 Feature Detection

SIFT is used as the main feature detector, with ORB available as an alternative. The purpose is to find distinctive points such as corners and textured regions that can be recognized in more than one image.

8.2 Feature Matching

Descriptors from two images are compared to find likely correspondences. A ratio-based filtering step removes ambiguous matches before geometric estimation.

8.3 Homography and RANSAC

For images that can be related by a planar projective transformation, a homography maps points from one image to another. RANSAC is used so that incorrect feature matches do not dominate the estimated transformation.

8.4 Homography Validation

The implementation does more than count RANSAC inliers. It checks the inlier ratio, reprojection error, spatial distribution of inliers, projected image geometry and canvas safety. This is important because a homography can have a low average error while still producing an extreme projection outside the useful image area.

8.5 Warping and Blending

Each accepted source image is projected into a common coordinate system. The overlapping regions are then combined to form the final panorama. The design avoids repeatedly warping an already-created panorama.

9. Project Structure

- main.py – command-line and GUI entry point.
- ordering.py – pairwise relationships and automatic image ordering.
- homography.py – homography estimation and geometric validation.
- matching/feature modules – feature extraction and descriptor matching.
- stitching/warping modules – image transformation and panorama composition.
- blending/output modules – final image combination and saving.
- test_homography.py – focused homography and validation tests.
- data/sample/ – sample images used for repeatable tests.
- outputs/ – generated panorama results.
- README.md / statement.md – project documentation and submission material.

10. Testing and Results

Testing was carried out using both prepared sample images and photographs taken from a real room. The project was also checked for cases where images have strong overlap and cases where the relationship between images is weak.

To test, run the given command :

```
python main.py -gui
```

it will open an interface from where you can choose your input file. Additionally the output will be saved with in the output folder in the project folder itself.

10.1 Real Room Input Images

The following photographs were supplied as a real-world test set. They show adjacent views of a desk area containing a bottle, lamp, laptop and surrounding wall/table. These images are useful for testing the system under uneven lighting and perspective changes.

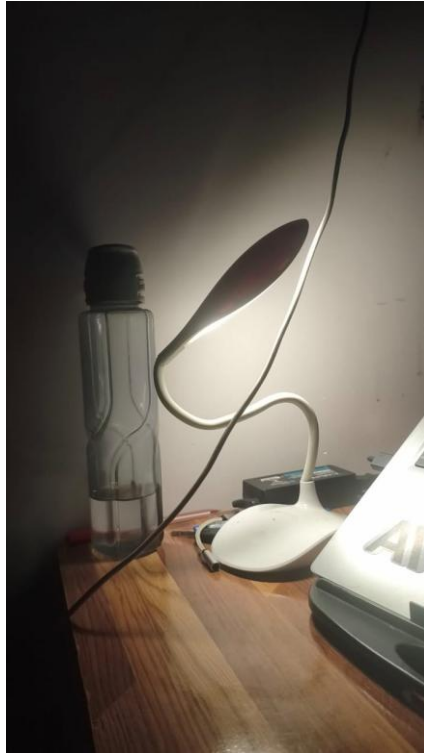


Figure 10.1. Real-world input image 1.



Figure 10.2. Real-world input image 2.



Figure 10.3. Real-world input image 3.



Figure 10.4. Real-world input image 4.

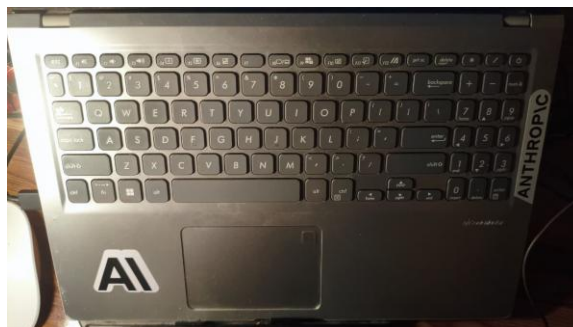


Figure 10.5. Real-world input image 5.

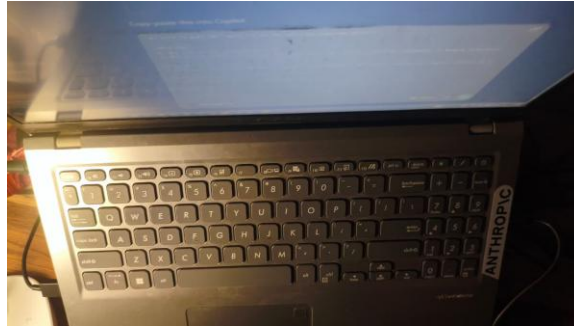


Figure 10.6. Real-world input image 6.



Figure 10.7. Real-world input image 7.

10.2 Sample Panorama Outputs

The supplied screenshots also show panorama outputs for several test scenes. The examples include mountain, landscape, night-sky and bridge sequences, along with the real-room result. The important point is that the input images are related views rather than unrelated photographs.



Figure 10.8. Test evidence / panorama result 1.



Figure 10.9. Test evidence / panorama result 2.



Figure 10.10. Test evidence / panorama result 3.



Figure 10.11. Test evidence / panorama result 4.



Figure 10.12. Test evidence / panorama result 5.



Figure 10.13. Test evidence / panorama result 6.



Figure 10.14. Test evidence / panorama result 7.



Figure 10.15. Test evidence / panorama result 8.



Figure 10.16. Test evidence / panorama result 9.



Figure 10.17. Test evidence / panorama result 10.

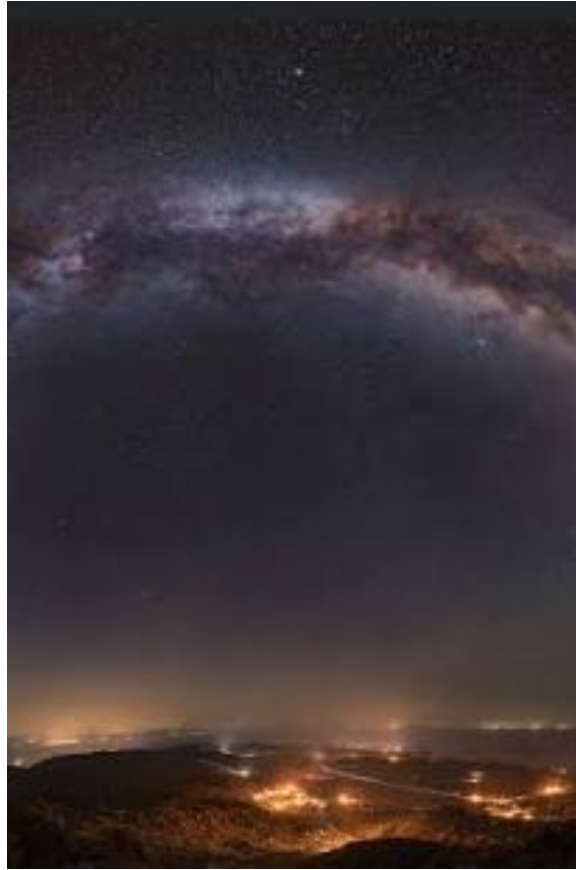


Figure 10.18. Test evidence / panorama result 11.



10.3 Representative Test Observations

Strongly overlapping sample images: The system is able to estimate transformations and produce a panorama.

Three-image real-room sequence: The feature matcher finds relationships, but the validation stage can reject a pair when the inliers are concentrated in a small region. This prevents a visibly distorted output.

Unrelated images: The system is intended to stop with an error rather than inventing a geometric relationship.

Large multi-image sets: Canvas-size and memory checks are important because chaining many perspective transforms can create a very large projected canvas.

Automatic ordering: Validated pairwise homographies are reused by the stitching stage so the same accepted geometric relationship is not unnecessarily estimated again.

11. Testing Approach

- Unit tests for homography estimation and validation.
- Synthetic three-image stitching test.
- Two-image pipeline test.
- Canvas safety test.
- Unrelated-image rejection test.
- Different-resolution input test.
- CLI help and argument validation tests.
- Manual testing through the GUI with real photographs.
- Incremental testing with 2–3 images before attempting larger image sets.

12. Challenges Faced

- Images taken from slightly different viewpoints can produce perspective distortion.
- A high number of feature matches does not automatically mean that the estimated homography is safe.
- Inlier points can be concentrated in a small part of an image, which can make a transformation unstable for the full image.
- Large chains of images can create a very large projected canvas and increase memory consumption.
- Real photographs have uneven illumination, reflections and regions with very few distinctive features.
- Filename order cannot always be assumed to represent the physical left-to-right order of photographs.

13. Design Decisions and Rationale

A classical computer vision approach was selected because panorama stitching directly demonstrates concepts covered in a Computer Vision course: local feature extraction, descriptor matching, robust estimation, projective geometry, image warping and image blending. SIFT was selected for its distinctive local features, while ORB was retained as a faster alternative.

Another important design choice was to validate the estimated homography instead of accepting every RANSAC result. In testing, this made the application more conservative, but it also reduced the chance of producing a completely distorted panorama from weak geometric evidence.

14. Learnings and Key Takeaways

The main learning from the project was that panorama stitching is not only about finding matching features. The geometric relationship between images has to be checked before an image is warped. I also learned that real photographs behave differently from clean sample images, especially when the camera moves too much, there is parallax, or the overlap contains little texture.

15. Future Enhancements

- Improve seam selection and exposure compensation for difficult lighting conditions.
- Add more advanced blending such as multiband blending.
- Improve handling of parallax-heavy scenes.
- Provide a visual match graph in the GUI.
- Add progress indicators for large image sets.
- Support more robust automatic cropping and panorama straightening.
- Add a benchmark section comparing SIFT and ORB on the same input sets.

16. Conclusion

The Intelligent Panorama Builder provides a complete classical computer vision pipeline for converting overlapping photographs into a larger panoramic view. The project includes image input, feature matching, RANSAC homography estimation, validation, automatic ordering, warping, blending, output generation and error handling. Testing with both sample scenes and real photographs helped identify the practical limits of image stitching and motivated the additional geometric and memory-safety checks.

17. References

- OpenCV documentation – feature detection, feature matching, homography and image warping.
- D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” International Journal of Computer Vision, 2004.
- M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” Communications of the ACM, 1981.
- Course material and laboratory concepts related to feature extraction, geometric transformation and image stitching.

Appendix A – Input / Output Evidence

All supplied evidence images are included in this report so that the evaluator can compare the source photographs with the generated results. The real-room photographs are included individually above, followed by the supplied test-result screenshots.